

From Lines to Branches -- Stepping into Hierarchies

Lecture 09

Data Structures and Algorithms

Part 1: Why Trees?

Limitations of Linear Structures

So far, every data structure we have studied is **linear** -- elements arranged one after another.

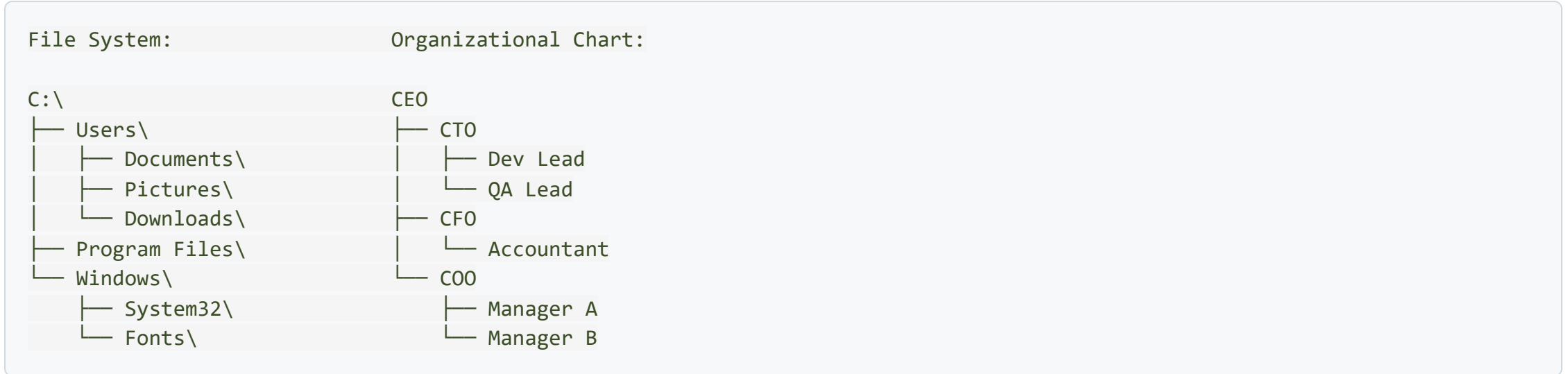
Structure	Organization
Arrays	Elements in contiguous memory slots
Linked Lists	Nodes chained by pointers
Stacks	LIFO access at one end
Queues	FIFO access at two ends

Problem: Not all real-world data is linear.

How do you represent a **file system**, an **organizational chart**, or the **HTML structure** of a web page using a linked list?

You cannot -- because these have **hierarchical** (parent-child) relationships.

Hierarchies Are Everywhere



These are not chains -- they are **branching structures** where one item can lead to many.

We need a data structure that captures this: the **Tree**.

What is a Tree?

A **tree** is a collection of nodes connected by edges, with the following rules:

- There is one special node called the **root** (the starting point)
- Every other node has **exactly one parent**
- There are **no cycles** (you cannot loop back to a visited node)

Two ways to define it:

Non-recursive: A connected, acyclic graph with a distinguished root node.

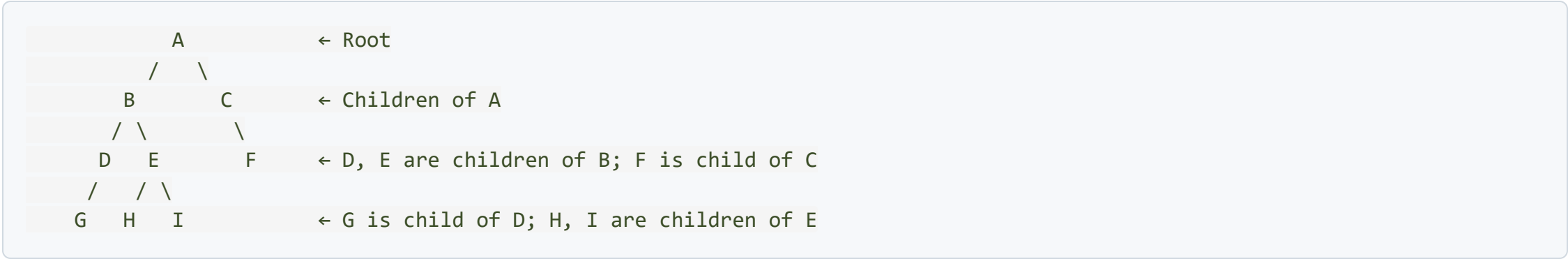
Recursive: A tree is either:

- Empty (no nodes), or
- A root node connected to zero or more **subtrees**, each of which is also a tree.

Analogy: Think of an actual tree in nature -- but **flipped upside down**. The root is at the top, and the leaves are at the bottom.

Part 2: Tree Terminology

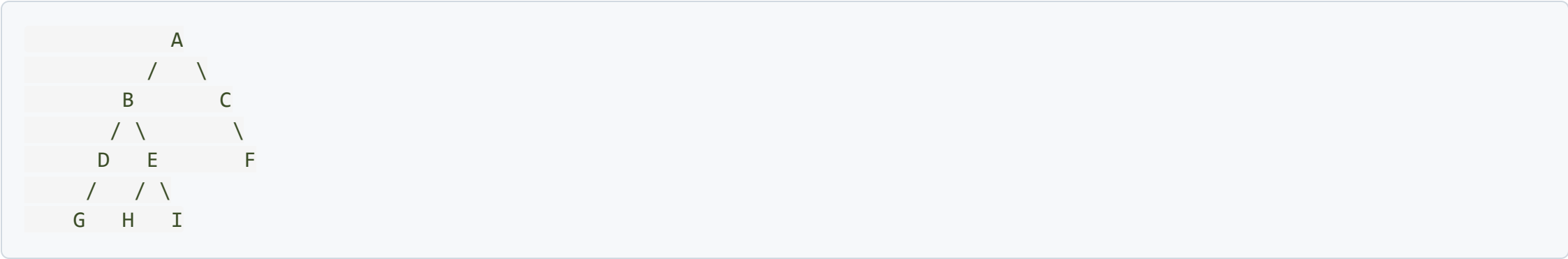
The Vocabulary of Trees



Term	Meaning	Example
Root	The topmost node (no parent)	A
Parent	The node directly above	B is parent of D, E
Child	A node directly below	D, E are children of B
Sibling	Nodes sharing the same parent	D and E are siblings
Leaf	A node with no children	G, H, I, F
Internal Node	A node with at least one child	A, B, C, D, E

More Vocabulary

Using the same tree:



Term	Meaning	Example
Edge	Connection between parent and child	A--B, B--D, etc.
Path	Sequence of edges from one node to another	A -> B -> E -> H
Degree of a Node	Number of children it has	Degree(B) = 2, Degree(F) = 0
Degree of the Tree	Maximum degree of any node	Degree(Tree) = 2
Subtree	A node and all its descendants	Subtree rooted at B = {B, D, E, G, H, I}

Height, Depth, and Level

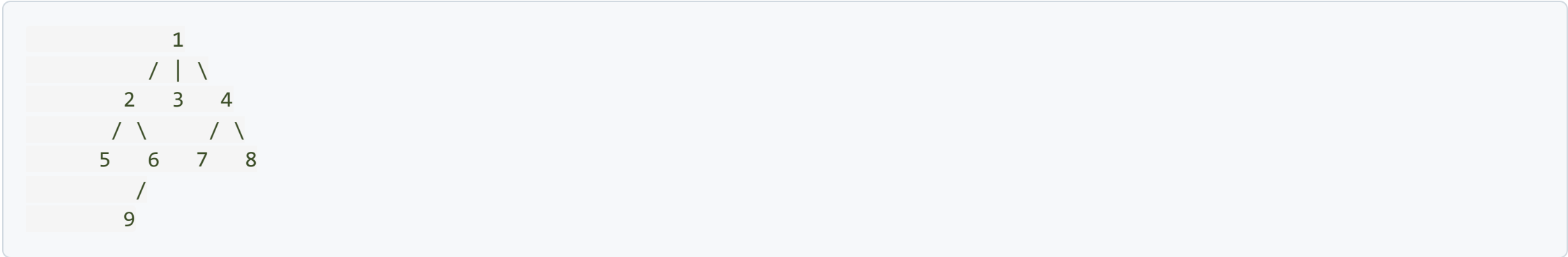
These three terms cause the most confusion. Let us be precise.



Term	Definition
Depth of a node	Number of edges from the root down to that node
Height of a node	Number of edges in the longest path from that node down to a leaf
Height of the tree	Height of the root node
Level	Same as depth (Level = Depth)

Key distinction: Depth is measured **downward from the root**. Height is measured **upward from the leaves**.

Quick Exercise: Compute Depth and Height



Fill in the table:

Node	Depth	Height	Degree
1	?	?	?
3	?	?	?
6	?	?	?
9	?	?	?

(Try this before moving on)

Exercise: Solution



Node	Depth	Height	Degree
1	0	3	3
3	1	0	0
6	2	1	1
9	3	0	0

Height of the tree = Height of root = **3**

Degree of the tree = max degree of any node = **3** (at node 1)

Part 3: Types of Trees

General Tree vs Binary Tree

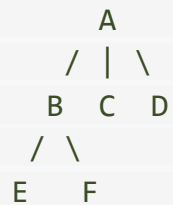
General Tree

- Each node can have **any number** of children
- No particular ordering among siblings
- Example: file system directories

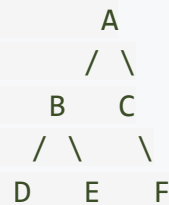
Binary Tree

- Each node has **at most 2 children**: a **left child** and a **right child**
- The distinction between left and right **matters** -- they are not interchangeable
- The simplest non-linear data structure
- Foundation for: BSTs, Heaps, Expression Trees, Huffman Trees, and more

General Tree:



Binary Tree:

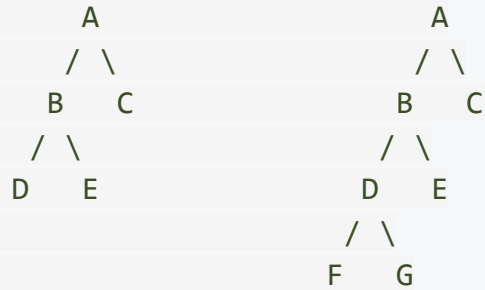


From here on, we focus on binary trees.

Special Types of Binary Trees

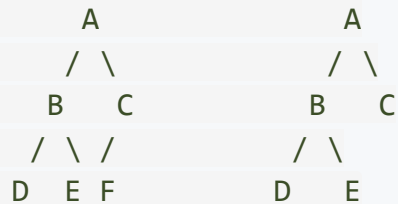
Full (Strict) Binary Tree

Every node has **either 0 or 2 children** -- no node has exactly 1 child.



Complete Binary Tree

All levels are fully filled **except possibly the last**, which is filled **from left to right**.

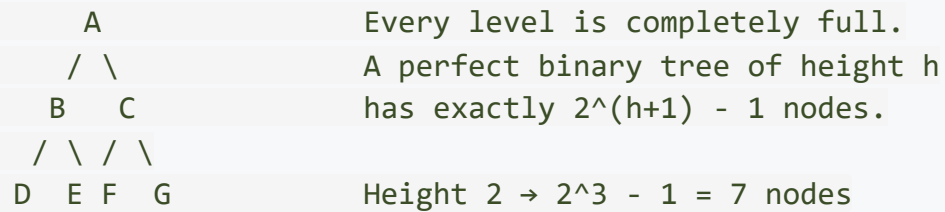


Left: complete. Right: also complete (last level fills left to right).

More Special Types

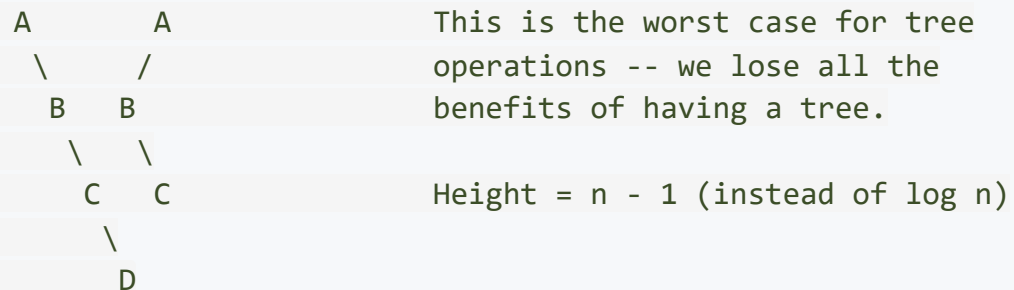
Perfect Binary Tree

All internal nodes have exactly 2 children, and **all leaves are at the same level**.



Degenerate (Pathological) Binary Tree

Every internal node has **only 1 child** -- this is essentially a **linked list**!



Summary of Binary Tree Types

Type	Rule	Example Use
Full	Every node has 0 or 2 children	Expression trees
Complete	All levels full except last (left-filled)	Binary heaps
Perfect	All levels completely full	Theoretical analysis
Degenerate	Every node has 0 or 1 child	Worst-case BST

Relationships:

- Every **perfect** binary tree is both **full** and **complete**
- A **complete** binary tree is not necessarily full (last level may not be full)
- A **full** binary tree is not necessarily complete (leaves can be at different levels)

Part 4: Properties of Binary Trees

Key Mathematical Properties

These formulas come up frequently in analysis and problem-solving.

Property 1: Maximum Nodes at a Level

At level l , the maximum number of nodes is:

$$\text{max nodes at level } l = 2^l$$

Level 0:	1 node (root)	= $2^0 = 1$
Level 1:	2 nodes	= $2^1 = 2$
Level 2:	4 nodes	= $2^2 = 4$
Level 3:	8 nodes	= $2^3 = 8$

Property 2: Maximum Nodes in a Tree of Height h

$$\text{max total nodes} = 2^{h+1} - 1$$

A tree of height 3 can have at most $2^4 - 1 = 15$ nodes.

More Properties

Property 3: Minimum Height for n Nodes

If we have n nodes and want the **shortest** (most balanced) tree:

$$h_{min} = \lfloor \log_2 n \rfloor$$

Example: 15 nodes can fit in a tree of height $\lfloor \log_2 15 \rfloor = 3$.

Property 4: NULL Pointers in a Binary Tree

A binary tree with n nodes has **$2n$ total pointers** (each node has left and right).

Only $n - 1$ are used as edges (since each non-root node has exactly one parent).

So the number of **NULL pointers** is:

$$\text{NULL pointers} = 2n - (n - 1) = n + 1$$

This means **more than half** of all pointers are wasted! (We will revisit this when we study threaded binary trees.)

The Leaf-Node Relationship

Property 5: Leaves vs Degree-2 Nodes

In any **non-empty** binary tree, if:

- n_0 = number of leaf nodes (degree 0)
- n_2 = number of nodes with 2 children (degree 2)

Then:

$$n_0 = n_2 + 1$$

There is always exactly one more leaf than there are degree-2 nodes.

Verification

A (degree 2)	$n_2 = 3$ (A, B, C)
/ \	$n_0 = 4$ (D, E, F, G)
B C (both degree 2)	
/ \ / \	$n_0 = n_2 + 1$
D E F G (all leaves)	$4 = 3 + 1 \checkmark$

This property holds for *any* binary tree, no matter how unbalanced.

Property Summary Table

Property	Formula	Example (h=3)
Max nodes at level l	2^l	Level 3: 8
Max nodes in tree of height h	$2^{h+1} - 1$	15
Min height for n nodes	$\lfloor \log_2 n \rfloor$	15 nodes: h=3
NULL pointers for n nodes	$n + 1$	15 nodes: 16 NULLs
Leaf vs degree-2 relation	$n_0 = n_2 + 1$	8 leaves, 7 degree-2

Why do these matter?

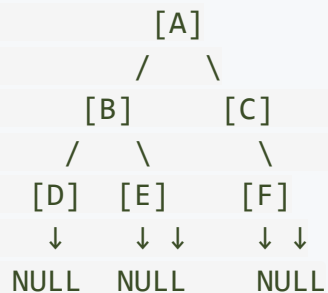
- **Height determines performance:** Operations on trees are typically $O(h)$
- **Balanced trees keep height at $O(\log n)$** -- the best case
- **Degenerate trees have height $O(n)$** -- the worst case
- Understanding these properties helps us reason about efficiency

Representing Binary Trees in Memory

Option 1: Linked Representation (Node-based)

Each node is a structure with three fields:

```
struct Node {  
    int data;  
    struct Node* left; // pointer to left child  
    struct Node* right; // pointer to right child  
};
```



Each box is a struct in memory.
NULL pointers for missing children.

Pros: Dynamic, easy to insert/delete, memory-efficient for sparse trees.

Cons: Extra memory for pointers, no random access.

Array Representation

Option 2: Array-based (for Complete Binary Trees)

Store nodes **level by level** in an array. For a node at index i (0-indexed):

Relationship	Formula
Parent	$\lfloor (i - 1) / 2 \rfloor$
Left Child	$2i + 1$
Right Child	$2i + 2$

Tree:

Array:

A

/ \

B C

/ \ /

D E F

Index: 0 1 2 3 4 5

Value: [A] [B] [C] [D] [E] [F]

Node C (index 2): left child = $2*2+1 = \text{index } 5 = F$
parent = $(2-1)/2 = \text{index } 0 = A$

Pros: No pointer overhead, cache-friendly, perfect for heaps (we will use this later!).

Cons: Wasteful for sparse or skewed trees (many empty slots).

When to Use Which Representation?

Feature	Linked (Node-based)	Array-based
Best for	General binary trees	Complete binary trees
Memory	Proportional to actual nodes	May waste space on gaps
Access	Follow pointers ($O(h)$)	Direct index ($O(1)$)
Insert/Delete	Easy (pointer updates)	May require shifting
Cache performance	Poor (scattered in memory)	Good (contiguous)
Used in	BSTs, expression trees	Binary heaps, segment trees

Key insight:

- We will use the **linked representation** for BSTs and AVL trees (general shape)
- We will use the **array representation** for binary heaps (always complete)

Part 5: Why Trees Matter -- The Big Picture

Linear vs Hierarchical: A Comparison

Aspect	Linear (Array/List)	Hierarchical (Tree)
Relationship	One predecessor, one successor	One parent, many children
Search (sorted)	$O(n)$ or $O(\log n)$ with binary search	$O(\log n)$ with BST
Insert (sorted)	$O(n)$ -- must shift elements	$O(\log n)$ with BST
Structure	Flat	Branching
Examples	Shopping list, queue at counter	File system, family tree

The promise of trees: We can achieve $O(\log n)$ for **both** search and insert -- something no linear structure can do efficiently.

This is the core motivation for Binary Search Trees, which we will study in Lecture 11.

What Lies Ahead: The Tree Family

This Unit (8 Lectures):

- └─ Lecture 09: Introduction to Trees (this lecture)
- └─ Lecture 10: Binary Tree Traversals + Threaded Binary Trees
- └─ Lecture 11: Binary Search Trees (search, insert, delete)
- └─ Lecture 12: Balanced BSTs + AVL Tree Rotations
- └─ Lecture 13: AVL Deletion + Red-Black Trees
- └─ Lecture 14: Multi-Way Search Trees + B-Trees
- └─ Lecture 15: B-Tree Operations + Applications
- └─ Lecture 16: Binary Heaps + Heap Sort

Each lecture builds directly on what we learn today:

- **Traversals** rely on the tree structure and terminology
- **BSTs** add an ordering rule to binary trees
- **AVL and Red-Black Trees** fix the degenerate tree problem
- **B-Trees** extend trees beyond binary for disk-based storage
- **Heaps** use the array representation of complete binary trees

Key Takeaways

1. A **tree** is a hierarchical data structure with a root node and no cycles
2. **Key terminology:** root, parent, child, sibling, leaf, internal node, edge, path, degree, height, depth, level
3. A **binary tree** has at most 2 children per node -- the simplest and most important non-linear structure
4. **Special types:** full, complete, perfect, and degenerate binary trees each have distinct properties
5. **Mathematical properties** help us reason about tree performance: height determines operation cost
6. **Two representations:** linked (for general trees) and array-based (for complete trees like heaps)
7. Trees promise $O(\log n)$ operations -- but only when **balanced** (more on this in later lectures)

Review Questions

1. What is the difference between **depth** and **height** of a node? Give an example.
2. A binary tree has 10 leaf nodes. How many nodes of degree 2 does it have?
3. What is the maximum number of nodes in a binary tree of height 5?
4. A complete binary tree has 31 nodes. What is its height?
5. Draw all possible **distinct binary trees** with 3 nodes. (Hint: there are 5.)
6. Given a binary tree with 20 nodes, how many NULL pointers does it contain?
7. In the array representation of a complete binary tree, if a node is at index 7 (0-indexed), what are the indices of its parent, left child, and right child?

Next Lecture

Binary Tree Traversals and Threaded Binary Trees

- Four ways to visit every node: In-order, Pre-order, Post-order, Level-order
- Reconstructing trees from traversal sequences
- Threaded binary trees: using wasted NULL pointers for efficient traversal

